AI for Lexical & Syntax Assistance

Name: Neelam Jashwanth

Roll Number: 24CSB0A49

Phase: Phase 2 (Design & Data Preparation)

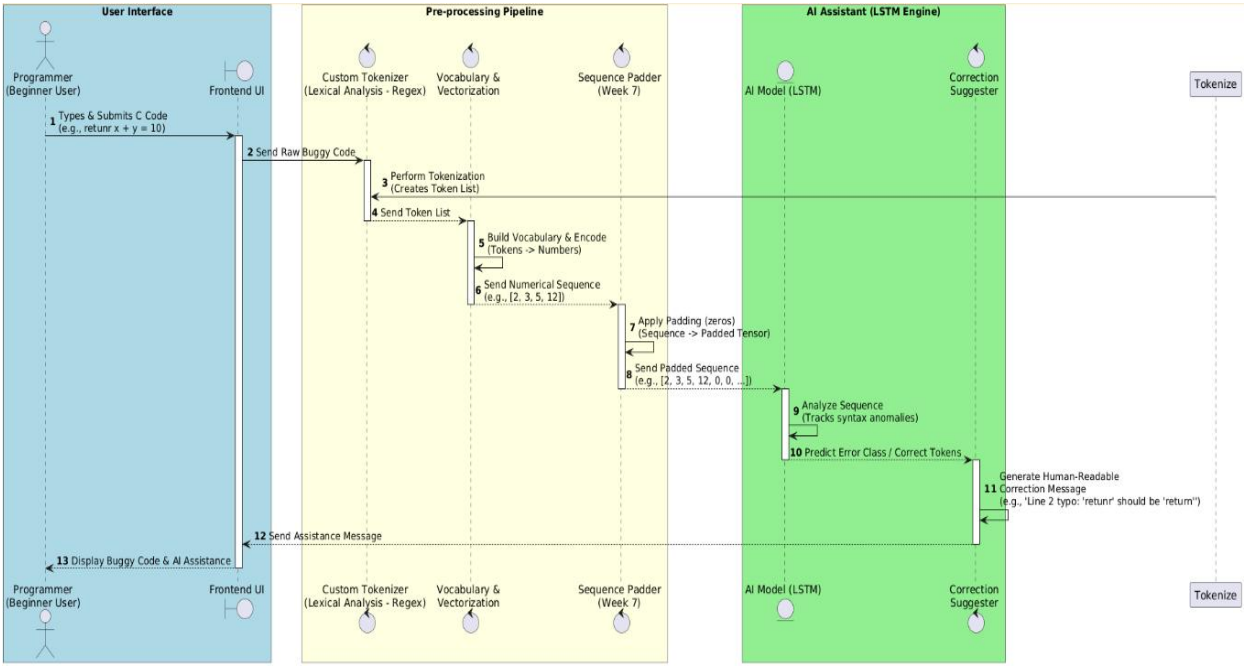
WEEK 4 REPORT: Architecture & Design Planning

1. Introduction

In Week 4, the focus is on designing the system architecture. This document outlines how the input code flows through the system, gets processed by the AI model, and results in a corrected code output. We have selected a modular architecture to ensure the system is scalable and easy to debug.

2. System Architecture

The system follows a "Pipeline Architecture." The raw code passes through sequential stages of processing.



Description of Architecture:

I. **Input Module:** Accepts the raw C/Python source code from the user.
II. **Tokenizer:** Converts the raw text into a sequence of meaningful tokens (keywords, identifiers, operators). It filters out comments and whitespace.
III. **AI Model (The Brain):** A Sequence-to-Sequence (Seq2Seq) LSTM model takes the "Incorrect Token Sequence" and predicts the "Correct Token Sequence."
IV. **Post-Processing:** Converts the predicted tokens back into readable source code.
V. **Output Module:** Displays the corrected code to the user.

3. Module Interaction Design

The system is divided into three major subsystems:

○ Module 1: Preprocessing Unit

- *Responsibility:* Cleaning data and Tokenization.
- *Input:* Raw Text File.
- *Output:* Numerical Vectors (Token IDs) for the AI.

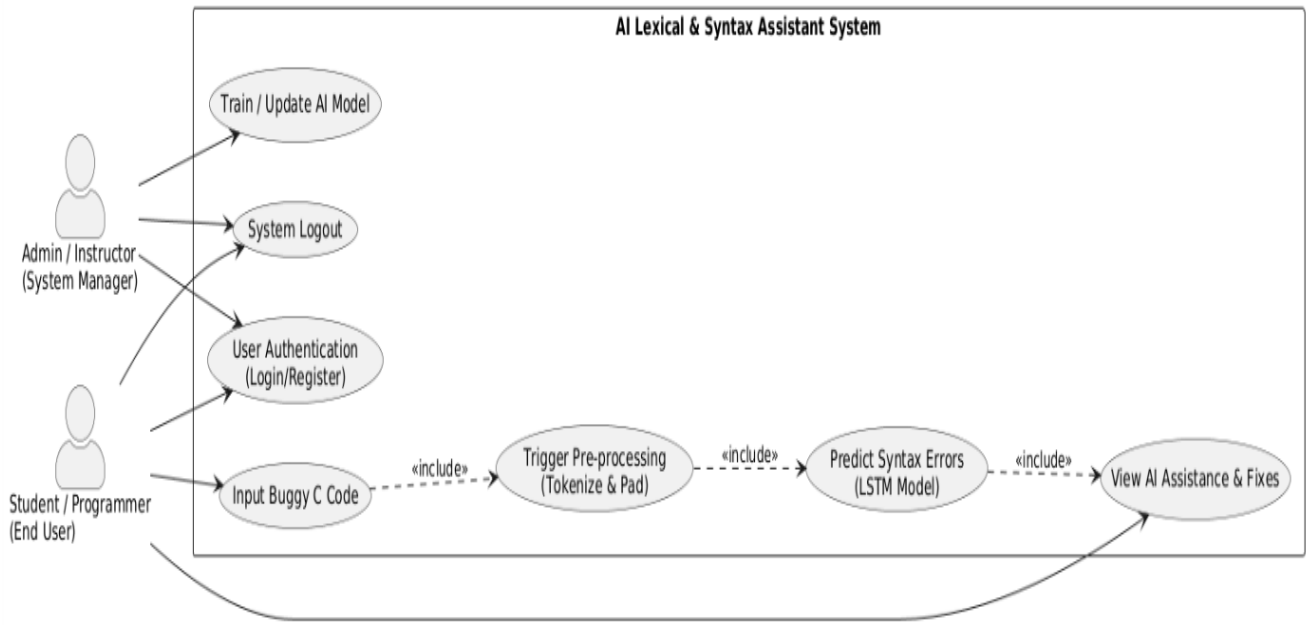
○ Module 2: The Core AI Engine

- ***Responsibility:*** Pattern recognition and Error Correction.
- ***Algorithm:*** Long Short-Term Memory (LSTM) Network with Attention Mechanism.
- **Input:** Vector Sequence.
- Output: Corrected Vector Sequence.

○ Module 3: Interface & Feedback

- **Responsibility***:* User Interaction (CLI/GUI).
- **Input:** User commands.

- **Output:** Final Code and Error Report**.**



4. Technology Stack Justification

We have selected the following tools for the implementation:

Component	Technology Selected	Reason for Selection
Language	**Python 3.8+**	Python has the best libraries for AI (TensorFlow) and Text Processing (NLTK/Regex).
AI Framework	**TensorFlow / Keras**	Provides built-in support for LSTM and Seq2Seq models, making implementation faster.

Component	Technology Selected	Reason for Selection
Model Type	**LSTM (RNN)**	Standard Neural Networks cannot understand the "order" of words. LSTM is specifically designed for sequential data like Code.
Tokenizer	**Custom Regex Tokenizer**	Standard tokenizers often crash on syntax errors. A custom regex-based tokenizer is more robust for buggy code.
Platform	**Google Colab / Jupyter**	Provides free GPU support for training the AI model efficiently.

5. Conclusion

The system architecture has been finalized. The pipeline ensures that lexical and syntax errors are handled systematically. The selected technology stack (Python + LSTM) is well-suited for this text-correction problem.